Aula 4 - Construtores



🚀 Aula 04: O Nascimento do Objeto - Construtores

Projeto: FiapRide (Sistema de Mobilidade Urbana) + Seu Projeto Pessoal

Foco: Inicialização Obrigatória, Clean Code e Java

Nível: Aplicar

🎯 Objetivo da Aula

Na aula passada, você fez algo avançado sem perceber: ao criar a Ana, você digitou `new Passageiro("Ana Silva", "222.222.222-22")`. Lembra que, antes disso, nas primeiras aulas, nossos objetos nasciam "em branco" apenas com parênteses vazios `()` e nós preenchíamos os dados depois?

Nós fizemos essa mudança por um motivo muito importante. Pense bem: **Existe algum carro no mundo real que sai da fábrica sem chassi, ou que é registrado no sistema sem placa e modelo?** Não!

Hoje vamos formalizar essa técnica que você já experimentou. Você vai dominar o uso de **Construtores**, entendendo como ditar as regras de nascimento de um objeto, obrigando que ele já venha ao mundo com os dados mínimos e essenciais para existir de forma válida.

Conceitos Chave:

- **Construtor Padrão (Default):** Aquele `()` vazio que o Java cria para você em segredo quando você não define regras.
- **Construtor Customizado:** A regra de nascimento que VOCÊ define (como fizemos com o Passageiro).
- **Estado Inicial:** Como o objeto deve estar configurado no exato milissegundo em que é criado na memória.

📖 A Teoria: O Certificado de Nascimento

Quando uma pessoa nasce no Brasil, o cartório não emite uma certidão em branco para a mãe preencher depois. O cartório **exige** informações obrigatórias: Nome da criança, Nome da Mãe e Data de Nascimento.

Na programação, o **Construtor** é o cartório. É um método especial que é executado **uma única vez** durante a vida do objeto: exatamente no momento do `new`.

Regras de Ouro de um Construtor em Java:

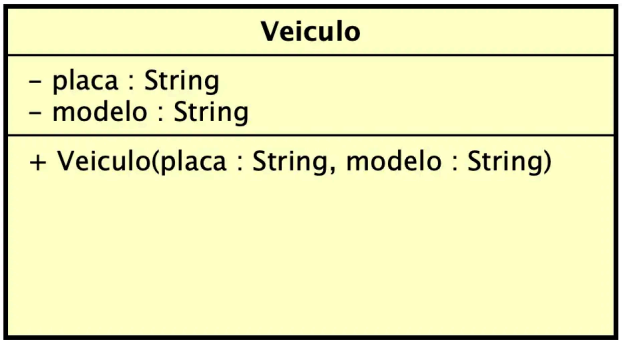
1. Ele tem **exatamente o mesmo nome da Classe** (com letra maiúscula).
2. Ele **NÃO tem tipo de retorno** (nem mesmo `void`).

🛠️ Parte 1: Modelagem Visual (Astah)

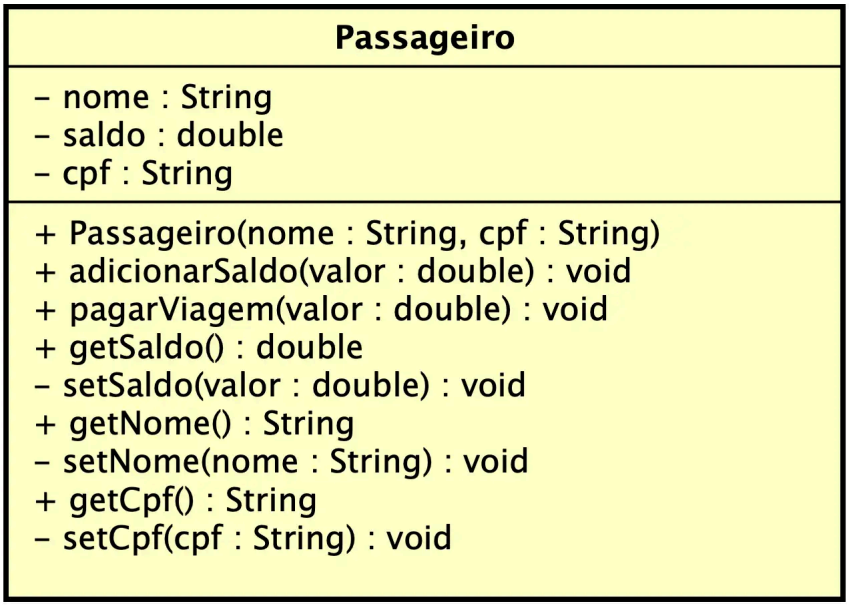
Vamos modelar a classe `Veiculo` do nosso FiapRide.

1. Abra o **Astah**.
2. Crie uma nova classe chamada `Veiculo`.
3. Adicione os atributos privados (`-`): `placa: String` e `modelo: String`.
4. Adicione uma Operação (`+`). O nome dela deve ser exatamente `Veiculo`.
5. Coloque os parâmetros dentro dos parênteses: `Veiculo(placa: String, modelo: String)`.

No UML, o construtor se parece com um método normal, mas o nome idêntico ao da classe o denuncia.



Classe Passageiro atualizada:



🪄 Dica de Mestre: Exportando Código Java direto do Astah

Você sabia que não precisa digitar toda a estrutura da classe do zero no Eclipse? O Astah é uma ferramenta inteligente e consegue ler o seu diagrama UML para gerar o "esqueleto" do código Java automaticamente!

Isso economiza tempo, mas lembre-se: ele gera apenas a estrutura (os atributos e a assinatura dos métodos). A lógica de negócio (os `ifs`, as regras do construtor) você ainda terá que programar.

Passo a Passo da Exportação:

1. Com o seu diagrama pronto e salvo no **Astah**, olhe para o menu superior.
2. Clique em **Tools** (Ferramentas).
3. Vá em **Java** e selecione a opção **Export Java...**
4. **Selecionar Local:** Na janela que abrir, escolha a pasta onde o arquivo `.java` será salvo (Dica: você pode selecionar diretamente a pasta `src/br/com/fiapride/model` do seu projeto no Eclipse, ou salvar na Área de Trabalho e arrastar depois).
5. **Selecionar Diagrama e Classe:** Marque as caixinhas correspondentes ao diagrama e à classe que você quer exportar (Ex: `Veiculo`).
6. Clique no botão **OK**.
7. Se aparecer uma janela de confirmação perguntando se deseja continuar, clique em **Yes**.

⚠️ O Toque do Engenheiro (Atenção ao Clean Code!)

Vá para o Eclipse, atualize seu projeto (F5) e abra a classe gerada.

O Astah gera o código exatamente como você desenhou. Se você esqueceu de colocar um atributo como Privado (`-`) no desenho, ele nascerá Público (`public`) no código! Sempre revise o código gerado para garantir que o **Encapsulamento** e as regras do seu **Construtor** estão corretos.

💻 Parte 2: O Desafio FiapRide (Mão na Massa)

Vamos programar a nossa classe `Veiculo` garantindo que nenhum "carro fantasma" (sem placa) rode no nosso aplicativo.

Passo 1: Criando a Classe e o Construtor

No Eclipse, dentro do pacote `br.com.fiapride.model`, crie a classe `Veiculo`.

```
1  package br.com.fiapride.model;
2
3  public class Veiculo {
4
5      // 1. Atributos sempre privados (Encapsulamento)
6      private String placa;
7      private String modelo;
8
9      // 2. O CONSTRUTOR (O Cartório)
10     public Veiculo(String placa, String modelo) {
11         // Usamos o setter privado para validar a placa logo no nascimento!
12         this.setPlaca(placa);
13         this.modelo = modelo;
14         System.out.println("🚗 Registro inicial: Um " + this.modelo + " nasceu com a
   placa " + this.placa);
15     }
16
17     // 3. Getters (Apenas leitura)
18     public String getPlaca() {
19         return this.placa;
20     }
21
22     public String getModelo() {
23         return this.modelo;
24     }
25
26     // 4. COMPORTAMENTO PÚBLICO (A regra de negócio / Ação)
27     // Este é o método que o programa principal vai chamar. Ele representa uma ação
   real.
28     public void atualizarPlaca(String novaPlaca) {
29         System.out.println("🔄 Solicitada atualização de placa no Detran para o veíc
   ulo " + this.modelo + "...");
30         this.setPlaca(novaPlaca); // Chama o método privado que contém a regra de se
   gurança
31     }
32
33     // 5. SETTERS PRIVADOS (A engrenagem interna de validação)
34     // O modificador é 'private'. Ninguém de fora da classe consegue usar.
35     private void setPlaca(String novaPlaca) {
```

```
        // Regra de negócio: a placa não pode ser nula nem vazia
        if (novaPlaca != null && !novaPlaca.trim().isEmpty()) {
            this.placa = novaPlaca;
            System.out.println("✅ Sucesso: A placa agora é " + this.placa);
        } else {
            System.out.println("❌ Erro de Validação: A placa informada é inválid
a!");
        }
    }

    // Obs: O modelo do carro é IMUTÁVEL na vida real (um Fiat Uno não se transforma
em um Corolla).
    // Portanto, a regra de negócio exige que NÃO exista um método setModelo()!
}
```

Passo 2: O Teste de Fogo (`SistemaPrincipal`)

Agora, vá até a sua classe `SistemaPrincipal` (ou onde você faz os testes) e tente criar o veículo.

```
package br.com.fiapride.main;

import br.com.fiapride.model.Veiculo;

public class SistemaPrincipal {
    public static void main(String[] args) {

        // TESTE 1: A forma correta (Nascimento legalizado)
        Veiculo meuCarro = new Veiculo("ABC-1234", "Toyota Corolla");

        // TESTE 2: Tentando burlar o sistema (Descomente a linha abaixo)
        // Veiculo carroFantasma = new Veiculo();
        // 🛑 ERRO DE COMPILAÇÃO!
        // O Java vai sublinhar de vermelho e dizer: "The constructor Veiculo() is u
ndefined".
       // Por quê? Porque assim que você cria um construtor com regras,
       // o Java DESTRÓI aquele construtor vazio padrão. Você é obrigado a passar a
placa!
   }
}
```

⚡ Microdesafio Pessoal: O Nascimento do seu Objeto

Vá para a classe do **seu** projeto pessoal (Garrafa, Pet, Smartphone, etc).

Sua Missão:

1. Crie um construtor para a sua classe.

2. Escolha de **1 a 3 atributos** que são absolutamente essenciais para o seu objeto existir.
   - *Ex: Um Pet não pode nascer sem* `nome` *e* `especie`*.*
3. Dentro do construtor, faça as atribuições (`this.setAtributo(parametro);`).
4. Vá na classe de teste (`TesteMeuObjeto`), apague o `new Objeto()` vazio que está dando erro e conserte passando os argumentos corretos.

🧼 Checklist: Clean Code (A Regra dos 3 Parâmetros)

Robert C. Martin (Uncle Bob) no livro *Clean Code* nos dá um alerta rigoroso sobre construtores e métodos:

- [ ] **Quantidade Ideal de Parâmetros:** O ideal é zero (se fizer sentido), um ou dois.
- [ ] **Limite Máximo:** Três parâmetros (`placa`, `modelo`, `ano`).
- [ ] **O Perigo:** Se o seu construtor tem 5, 6 ou 10 parâmetros, o código fica ilegível. Imagine isso: `new Veiculo("ABC", "Gol", 2020, "Preto", 4, 1.0, true)`. Quem ler não fará ideia do que significa aquele `true` ou aquele `4`.*(Nota: No futuro, aprenderemos Padrões de Projeto como o* ***Builder*** *para resolver objetos gigantes).*

🧠 Pergunta de Reflexão

> "Na nossa classe `Veiculo`, nós tomamos duas decisões arquitetônicas muito importantes:
>
> 1. Nós **não** criamos o método `setModelo()`.
> 2. O `setPlaca()` foi criado como **privado**, e criamos um método público chamado `atualizarPlaca()` para acessá-lo.
>
> Pensando no mundo real e no Clean Code: Por que é um erro gravíssimo clicar em 'Gerar Getters e Setters para tudo' automaticamente na sua IDE? Como as nossas duas decisões acima protegem o sistema de fraudes e falhas de lógica?"

Dica: Pense sobre o que pode ou não mudar fisicamente em um carro, e a diferença entre "alterar um dado no banco" e "executar um processo real no Detran".

📤 Finalizando

Não se esqueça de atualizar seu código no GIT!

```
git add .
git commit -m "feat: adiciona construtores as classes exigindo parametros obrigatorios"
git push origin main
```

Bons estudos!

